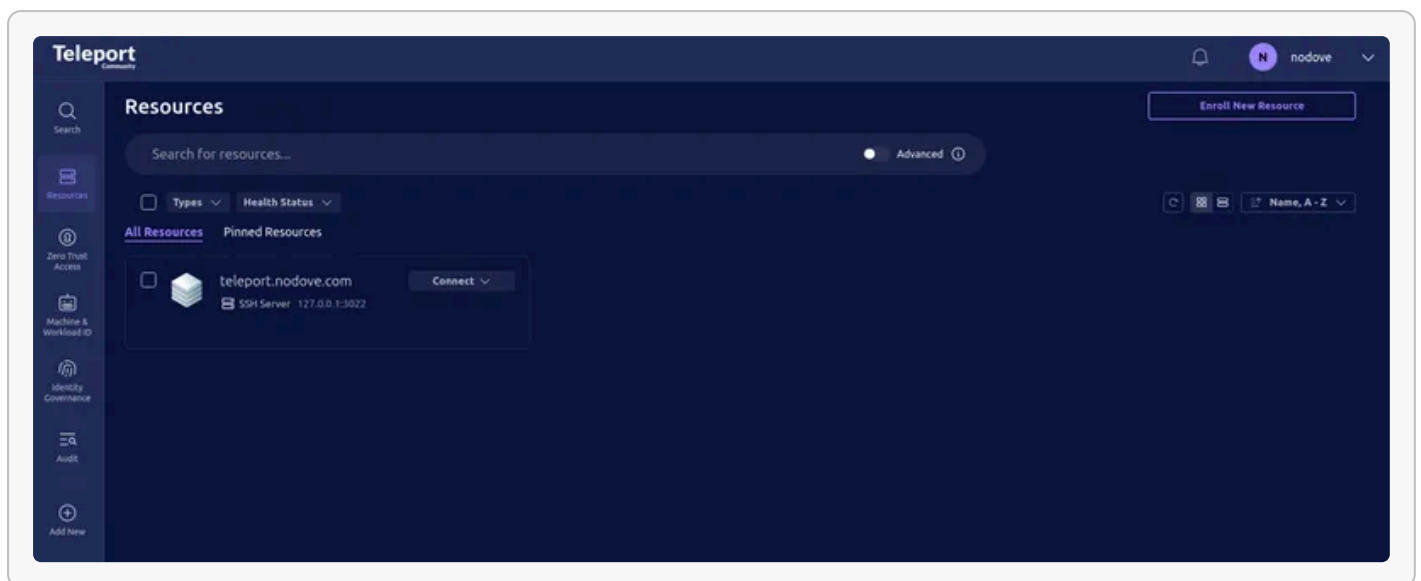


Teleport 설치와 서버 관리 가이드

SSH 키를 서버마다 흩뿌려 관리하는 방식은 서버 수가 늘어날수록 금방 피곤해집니다. Teleport는 웹 UI, 짧은 수명의 인증서, 세션 녹화, RBAC를 한데 묶어...

A 2026년 3월 1일 16분 읽기 Admin

#teleport #ssh #security #devops #infra #nginx #cloudflare #dns-01



teleport

SSH 키를 서버마다 흩뿌려 관리하는 방식은 서버 수가 늘어날수록 금방 피곤해집니다. Teleport는 웹 UI, 짧은 수명의 인증서, 세션 녹화, RBAC를 한데 묶어서 이 문제를 정리해 주는 도구입니다.

1. 도메인 연결과 포트 오픈까지 끝냈고, 바로 관리자 계정과 노드 등록을 진행하고 싶은 경우
2. Docker Compose로 Teleport Proxy/Auth 서버를 처음부터 구성하고 싶은 경우

어떤 템플릿으로 시작할지 먼저 고르기

이 글은 사실 두 가지 설치 템플릿을 같이 다룹니다. 서버의 443 포트를 Teleport에 바로 줄 수 있다면 아래의 기본 템플릿이 가장 단순합니다. 반대로 이미 다른 서비스가 80/443 를 점유하고 있거나, 인증서를 DNS API 기반으로 따로 관리하고 싶다면 뒤쪽의 실전 우회 템플릿을 쓰는 편이 훨씬 덜 고통스럽습니다.

템플릿	쓰는 상황	핵심 차이
기본 템플릿	Teleport가 443 을 직접 점유할 수 있을 때	<code>acme.enabled: true</code> 로 Teleport가 인증서를 직접 발급받고 갱신합니다.
실전 우회 템플릿	기존 서비스가 80/443 를 쓰고 있거나 DNS-01이 필요한 때	<code>Certbot DNS-01 + Nginx stream + https_keypairs</code> 조합으로 Teleport 앞단만 우회합니다.

빠른 시작

도메인 연결이 이미 끝난 상태라면 아래 순서로 진행하면 됩니다.

1. 첫 관리자 계정 생성

```
1 docker compose exec teleport tctl users add admin \  
2   --roles=editor,access \  
3   --logins=root,ubuntu,ec2-user
```

명령을 실행하면 1시간짜리 초대 URL이 출력됩니다. 그 링크를 브라우저에서 열고 비밀번호와 OTP를 등록하면 첫 관리자 계정 생성이 끝납니다.

2. 웹 UI 접속 확인

`https://your-domain.com`

443 포트는 기본 HTTPS 포트라서 브라우저에서는 보통 `:443` 을 생략해도 됩니다.

3. 로컬 PC에 tsh 설치

```
1 # macOS
2 brew install teleport
3
4 # Linux
5 curl https://goteleport.com/static/install.sh | bash -s 16.x oss
6
7 # Windows
8 # https://goteleport.com/download 에서 다운로드
```

4. 로컬에서 Teleport 로그인

```
tsh login --proxy=your-domain.com:443 --user=admin
```

브라우저 인증을 마치면 로컬에 Teleport 인증서가 저장됩니다. 기본 인증서 수명은 보통 12시간입니다. 뒤쪽 실전 우회 템플릿을 쓰는 경우에는 여기의 :443 만 :18443 같은 모의 외부 포트로 바꿔 주면 됩니다.

5. 노드 조인 토큰 발급

```
docker compose exec teleport tctl tokens add --type=node --ttl=1h
```

출력된 token 과 ca-pin 을 복사해 둡니다.

6. 대상 서버에 Agent 설치 및 등록

```
1 curl https://goteleport.com/static/install.sh | bash -s 16.x oss
2
3 sudo teleport configure \
4   --roles=node \
5   --proxy=your-domain.com:443 \
6   --token=<발급받은_토큰> \
7   --ca-pin=<발급받은_CA_PIN> \
8   --nodename=my-server-01 \
9   -o /etc/teleport.yaml
10
```

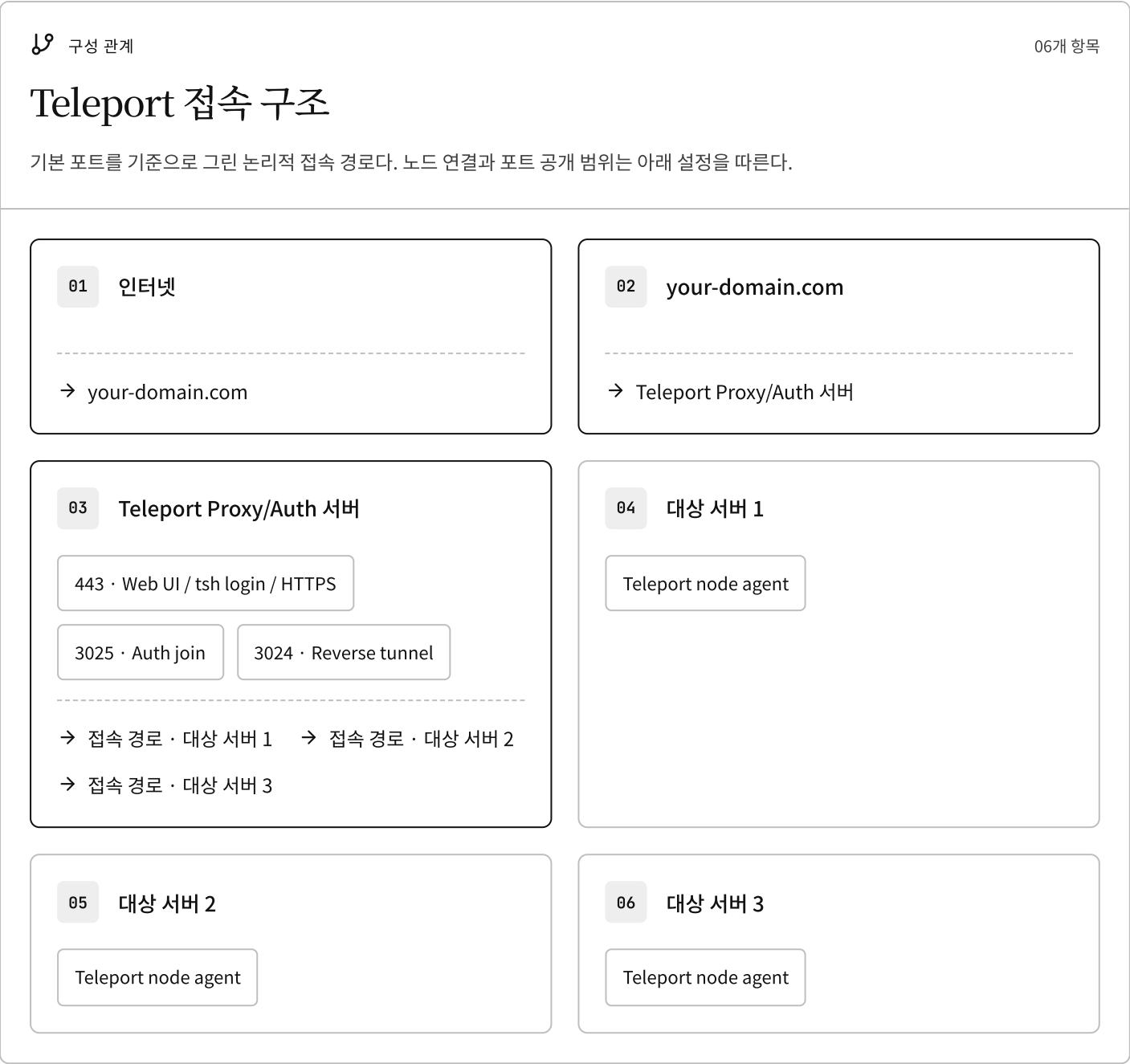
```
11 sudo systemctl enable teleport
12 sudo systemctl start teleport
```

이 단계 역시 실전 우회 템플릿을 쓰는 경우에는 `--proxy=teleport.example.net:18443` 처럼 외부에서 접속하게 될 모의 주소와 포트로 맞춰 주면 됩니다.

7. 등록과 접속 확인

```
1 tsh ls
2 tsh ssh ubuntu@my-server-01
```

여기까지 되면 기본 구성은 완료입니다. 아래부터는 처음부터 설치하는 전체 과정을 한 번에 정리한 버전입니다.



- **역할:** 실제로 접근하고 싶은 원격 서버(예: 웹 서버, DB 서버)입니다.
- **작업 내용:** 이 서버에는 Teleport 전체 데몬 대신 가벼운 **'Node Agent'**만 설치합니다. 에이전트는 발급받은 토큰을 사용해 Proxy 서버에 자신을 '등록'하고, 이후 프록시의 지시(SSH 세션)를 기다립니다.
- **로컬 PC (사용자 환경)**
 - **역할:** 개발자나 인프라 관리자가 실제 업무를 보는 노트북, 데스크톱입니다.
 - **작업 내용:** 클라이언트 도구인 **tsh** 를 설치하고, **tsh login** 으로 중앙 서버에 인증을 받아 짧은 수명의 인증서를 획득합니다. 이후 **tsh ssh** 명령을 통해 원하는 대상 서버로 접속합니다.

운영 흐름은 단순합니다.

1. 사용자는 **tsh login** 으로 짧은 수명의 인증서를 발급받습니다.
2. 각 서버는 Teleport Node Agent로 클러스터에 등록됩니다.
3. 실제 SSH 연결은 Teleport Proxy를 통해 중계됩니다.
4. 세션은 중앙에서 기록되고, 권한은 Role로 통제됩니다.

실전 예시: 80/443 점유 환경에서 DNS-01 + Nginx L4로 우회한 구성

여기서부터는 제가 실제로 부딪혔던 케이스를 기준으로 정리합니다. 다만 도메인과 외부 포트는 모두 설명용 모의값으로 바꿔 두었습니다. 서버에는 이미 다른 서비스가 **80/443** 을 쓰고 있었고, 그 포트를 Teleport에 직접 넘길 수 없었습니다. 그렇다고 Teleport를 포기하기엔 아쉬웠기 때문에, 인증서는 Cloudflare DNS API를 이용한 **DNS-01** 로 따로 발급하고, Teleport 앞단에는 아주 얇은 **Nginx stream** 프록시만 두는 방식으로 우회했습니다.

핵심 아이디어는 단순합니다. 인증서 발급은 Teleport 내부 ACME에 맡기지 않고 **Certbot** 이 처리합니다. 외부에서 들어오는 **18443** 트래픽은 Nginx가 L4 계층에서 그대로 Teleport에 전달합니다. Teleport는 내부 **8443** 에서 인증서를 직접 읽고, **proxy_listener_mode: multiplex** 로 웹 UI와 **tsh login** , SSH, gRPC를 함께 처리합니다. 즉, Teleport의 기능은 그대로 쓰되, 앞단 인증서와 포트 충돌만 분리해서 다루는 셈입니다.

 항목 비교

04개 항목

Nginx를 거치는 포트 연결

01 외부 18080 → Nginx 80

HTTPS 주소 `https://teleport.example.net:18443`으로 리다이렉트한다.

02 외부 18443 → Nginx 443

Teleport 8443으로 TLS 연결을 전달한다.

03 내부 3025

Teleport Auth join

04 내부 3024

Teleport reverse tunnel

템플릿 1. 작업 디렉터리와 Cloudflare 시크릿 준비

```
1 mkdir -p /opt/teleport-l4/{config,data,logs,nginx,certbot/conf,certbot/www}
2 cd /opt/teleport-l4
3
4 cat <<'EOF' > ./certbot/cloudflare.ini
5 dns_cloudflare_api_token=REPLACE_WITH_REAL_TOKEN
6 EOF
7
8 chmod 600 ./certbot/cloudflare.ini
```

Cloudflare 토큰은 최소한 해당 Zone의 DNS 수정 권한 정도만 주는 편이 안전합니다. 이 파일은 Certbot이 민감한 시크릿으로 취급하므로 600 권한이 아니면 실행 단계에서 바로 거절당할 수 있습니다.

템플릿 2. docker-compose.yml

```
1 services:
2   teleport:
3     image: public.ecr.aws/gravitational/teleport:18.7.1
4     container_name: teleport
```

```
5 restart: unless-stopped
6 hostname: teleport-proxy
7 volumes:
8   - ./config/teleport.yaml:/etc/teleport/teleport.yaml:ro
9   - ./data:/var/lib/teleport
10  - ./logs:/var/log/teleport
11  - ./certbot/conf:/etc/letsencrypt:ro
12 ports:
13   - "3025:3025"
14   - "3024:3024"
15 networks:
16   - proxy-net
17 ulimits:
18   nofile:
19     soft: 65536
20     hard: 65536
21 healthcheck:
22   test: ["CMD", "tctl", "status"]
23   interval: 30s
24   timeout: 10s
25   retries: 3
26 command: teleport start --config=/etc/teleport/teleport.yaml
27
28 nginx-teleport:
29   image: nginx:alpine
30   container_name: nginx-teleport
31   restart: unless-stopped
32   ports:
33     - "18080:80"
34     - "18443:443"
35   volumes:
36     - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
37   networks:
38     - proxy-net
39
40 certbot:
41   image: certbot/dns-cloudflare
42   container_name: certbot
43   volumes:
44     - ./certbot/conf:/etc/letsencrypt
45     - ./certbot/cloudflare.ini:/secrets/cloudflare.ini:ro
46   entrypoint /bin/s - 'trap exit TERM; while :; do sleep 12h & wait $$(!); certbot
47     t:          h          c renew; done'
48
49 networks:
50   proxy-net:
51     driver: bridge
```


여기서 중요한 부분은 Teleport가 더 이상 호스트의 443 을 직접 바인딩하지 않는다는 점입니다. 3024 와 3025 만 Teleport가 직접 열고, 실제 외부 진입점은 nginx-teleport 컨테이너가 18443 으로 받습니다. certbot 은 평소에는 잠자고 있다가 주기적으로 갱신만 확인합니다. 최초 인증서 발급은 아래에서 별도의 1회성 명령으로 강제로 실행합니다.

템플릿 3. ./nginx/nginx.conf

```
events {
    worker_connections 1024;
}

stream {
    upstream teleport_backend {
        server teleport:8443;
    }

    server {
        listen 443;
        proxy_pass teleport_backend;
        ssl_preread on;
    }
}

http {
    include      /etc/nginx/mime.types;
    default_type application/octet-stream;

    server {
        listen 80;
        server_name teleport.example.net;

        location / {
            return 301 https://$host:18443$request_uri;
        }
    }
}
```

이 설정의 목적은 "가공하지 않고 그대로 전달하는 것"입니다. stream {} 블록에서 ssl_preread on; 을 켜 두면 Nginx가 TLS를 종료하지 않고도 SNI를 읽을 수 있고, 실제 암호화와 멀티플렉싱은 Teleport가 맡게 됩니다. 만약 여기서 http {} 프록시처럼 처리해 버리면 Teleport 특유의 프록시 프로토콜 흐름이 중간에서 깨질 수 있습니다.

템플릿 4. `./config/teleport.yaml`

```
1 teleport:
2   nodename: teleport.example.net
3   data_dir: /var/lib/teleport
4   log:
5     output: /var/log/teleport/teleport.log
6     severity: INFO
7
8   auth_service:
9     enabled: true
10    cluster_name: "teleport.example.net"
11    listen_addr: 0.0.0.0:3025
12
13  proxy_service:
14    enabled: true
15    proxy_listener_mode: multiplex
16    web_listen_addr: 0.0.0.0:8443
17    public_addr: "teleport.example.net:18443"
18    acme:
19      enabled: false
20    https_keypairs:
21      - key_file: /etc/letsencrypt/live/teleport.example.net/privkey.pem
22        cert_file: /etc/letsencrypt/live/teleport.example.net/fullchain.pem
23
24  ssh_service:
25    enabled: true
26    labels:
27      env: production
28      role: control-plane
```

여기서는 `acme.enabled: false` 가 제일 중요합니다. 인증서를 Teleport가 직접 발급하려 들지 말고, 이미 Certbot 이 만들어 둔 경로를 읽기만 하도록 바꾸는 것입니다. `public_addr` 도 반드시 실제 외부 접속 포트와 맞아야 합니다. 이 값을 `443` 으로 두고 실제로는 `18443` 으로 서비스하면 `tsh login` 단계에서 곧바로 헛갈리기 시작합니다.

템플릿 5. 최초 인증서 발급

```
1 docker compose run --rm --entrypoint certbot certbot certonly \
2   --dns-cloudflare \
3   --dns-cloudflare-credentials /secrets/cloudflare.ini \
4   --dns-cloudflare-propagation-seconds 20 \
5   -d teleport.example.net \
6   --email ops@example.com \
```

```
7 --agree-tos \  
8 --no-eff-email
```

이 단계는 꼭 한 번은 수동으로 실행해 두는 편이 낫습니다. `certbot` 서비스는 기본적으로 갱신 루프를 돌도록 해 두었기 때문에, 그대로 `docker compose up` 만 하면 아무 인증서도 없는 상태에서 Teleport가 먼저 뜨거나, 반대로 Certbot은 갱신할 대상이 없어서 조용히 끝나는 애매한 상황이 생길 수 있습니다.

템플릿 6. 인증서 권한 문제 해결

```
1 sudo chmod -R 755 ./certbot/conf/archive  
2 sudo chmod -R 755 ./certbot/conf/live  
3 sudo chmod 644 ./certbot/conf/archive/teleport.example.net/privkey*.pem
```

실제로 가장 오래 붙잡고 있던 부분 중 하나가 여기였습니다. Certbot이 만든 개인키는 기본적으로 root 전용에 가깝게 만들어지기 때문에, 컨테이너 안의 Teleport 프로세스가 그대로 읽으려 하면 `permission denied` 로 끝나는 경우가 많습니다. 더 엄격하게는 ACL이나 그룹 소유권으로 푸는 편이 좋지만, 홈랩에서 먼저 확인해야 할 것은 "지금 Teleport가 이 파일을 읽을 수 있는가" 입니다.

템플릿 7. 기동과 실제 접속 확인

```
1 docker compose up -d  
2 docker compose logs -f teleport  
3  
4 tsh login --proxy=teleport.example.net:18443 --user=admin  
5 tsh ls
```

노드 등록도 같은 포트를 따라갑니다.

```
1 sudo teleport configure \  
2 --roles=node \  
3 --proxy=teleport.example.net:18443 \  
4 --token=<발급받은_토큰> \  
5 --ca-pin=<발급받은_CA_PIN> \  
6 --nodename=my-server-01 \  
7 -o /etc/teleport.yaml
```

결국 기본 템플릿과 달라지는 부분은 세 곳뿐입니다. 인증서 발급 방식이 HTTP-01 에서 DNS-01 으로 바뀌고, Teleport 앞에 L4 프록시가 하나 추가되며, 사용자가 붙는 외부 포트가 443 대신 18443 으로 바뀝니다. 관리자 계정 생성, `tctl tokens add`, `tsh ls`, `tsh ssh` 같은 나머지 운영 루틴은 그대로 재사용하면 됩니다.

실제로 막혔던 지점과 해결 방식

포트 충돌은 가장 먼저 마주치는 문제였습니다. 이미 다른 서비스가 80/443 을 점유하고 있으면 Teleport의 기본 ACME 템플릿은 바로 부딪힙니다. 이때 억지로 같이 붙이기보다, Teleport는 내부 8443 으로 내리고 외부에는 Nginx stream을 두는 쪽이 구조가 훨씬 단순했습니다.

두 번째는 인증서 발급 방식이었습니다. Teleport의 내부 ACME는 편하지만, 외부 443 이 Teleport 자신에게 곧바로 와야 제대로 동작합니다. 이 전제가 깨지는 순간에는 미련 없이 `acme.enabled: false` 로 바꾸고, Certbot DNS-01 로 인증서를 먼저 받아 넣는 편이 훨씬 예측 가능했습니다.

세 번째는 권한 문제였습니다. 인증서가 발급됐는데도 Teleport가 못 읽으면 겉으로는 "인증서가 있는데 왜 안 뜨지?"처럼 보여서 더 헷갈립니다. 이때는 컨테이너 안의 Teleport가 실제로 `/etc/letsencrypt/live/...` 아래 파일을 읽을 수 있는지부터 확인하는 게 가장 빠릅니다.

마지막은 클라이언트 포트 일관성이었습니다. 서버 설정만 18443 기준으로 바꾸고 `tsh login`, `teleport configure`, `public_addr` 중 하나라도 443 을 그대로 두면 바로 어딘가에서 꼬입니다. 실전에서는 "외부에서 사람들이 실제로 접속하는 주소와 포트를 모든 설정에 그대로 복사한다"는 원칙 하나가 제일 잘 먹었습니다.

사전 준비 체크리스트

시작 전에 아래 조건을 먼저 확인해 두면 중간에 막히는 일이 줄어듭니다.

- `your-domain.com` 이 Teleport 서버 IP를 가리키는 DNS A 레코드가 준비되어 있을 것
- 서버 인바운드 포트 443 , 3025 , 3024 가 열려 있을 것
- 선택적으로 3080 포트를 열어 HTTP를 HTTPS로 리다이렉트할 수 있을 것
- Teleport 서버에는 Docker와 Docker Compose가 설치되어 있을 것
- 대상 서버에는 `sudo` 또는 root 권한이 있을 것

1. Teleport 서버 디렉터리 준비

```
1 mkdir -p /opt/teleport/{config,data,logs}
2 cd /opt/teleport
```

구조는 아래처럼 가져가면 관리하기 편합니다.

🔗 구성 관계

06개 항목

/opt/teleport/ 디렉터리 구성

폴더와 파일의 포함 관계. 카드 아래에 하위 항목을 표시했다.

01 /opt/teleport/

- 하위 항목 · docker-compose.yml
- 하위 항목 · config/ → 하위 항목 · data/
- 하위 항목 · logs/

02 docker-compose.yml

03 config/

- 하위 항목 · teleport.yaml

04 teleport.yaml

05 data/

06 logs/

- config/ : 설정 파일
- data/ : 인증서, 세션 녹화, 상태 데이터
- logs/ : Teleport 로그

2. teleport.yaml 작성

```
1 teleport:
2   nodename: teleport-proxy
3   data_dir: /var/lib/teleport
4   log:
5     output: /var/log/teleport/teleport.log
6     severity: INFO
7
8   auth_service:
9     enabled: true
10    cluster_name: "your-domain.com"
11    listen_addr: 0.0.0.0:3025
12    tokens:
13      - "proxy,node,app,db:/var/lib/teleport/token"
14
15    proxy_service:
16      enabled: true
17      web_listen_addr: 0.0.0.0:443
18      public_addr: "your-domain.com:443"
19      https_keypairs: []
20      acme:
21        enabled: true
22        email: "admin@your-domain.com"
23
24    ssh_service:
25      enabled: false
```

teleport config — 원본 이미지

teleport config

핵심 포인트는 네 가지입니다.

- `cluster_name` : 클러스터 식별자입니다. 보통 실제 도메인과 맞춥니다.
- `public_addr` : 사용자가 접속하는 외부 주소입니다.
- `acme.enabled: true` : Let's Encrypt 인증서를 자동 발급하고 갱신합니다.
- `ssh_service.enabled: false` : 이 서버를 관리 대상 노드로 쓰지 않는다면 꺼 두는 편이 깔끔합니다. (저는 ssh 관리 서버로 사용하므로 `enabled: true` 설정해두었습니다.)

`acme.enabled: true` 를 쓰는 경우, 외부에서 443 포트로 정상 접근 가능해야 인증서 발급이 됩니다. 만약 이미 다른 서비스가 80/443 을 점유하고 있다면, 위의 실전 예시처럼 `DNS-01 + https_keypairs + Nginx stream` 조합으로

우회하는 편이 낫습니다.

3. `docker-compose.yml` 작성

```
1 # https://goteleport.com/docs/installation/docker/ 참고
2 services:
3   teleport:
4     image: public.ecr.aws/gravitational/teleport:18
5     container_name: teleport
6     restart: unless-stopped
7     hostname: teleport-proxy
8
9     volumes:
10      - ./config/teleport.yaml:/etc/teleport/teleport.yaml:ro
11      - ./data:/var/lib/teleport
12      - ./logs:/var/log/teleport
13
14     ports:
15      - "443:443"
16      - "3025:3025"
17      - "3024:3024"
18      - "3080:3080"
19
20     cap_add:
21      - NET_BIND_SERVICE
22
23     ulimits:
24       nofile:
25         soft: 65536
26         hard: 65536
27
28     healthcheck:
29       test: ["CMD", "tctl", "status"]
30       interval: 30s
31       timeout: 10s
32       retries: 3
33
```

이 설정에서 눈여겨볼 주요 옵션은 다음과 같습니다.

- `cap_add: [NET_BIND_SERVICE]` : 루트 권한 없이도 컨테이너가 1024번 이하의 포트(예: 443 포트)를 바인딩할 수 있도록 네트워크 권한을 부여합니다.

- `ulimits` : Teleport는 수많은 동시 세션과 연결을 중계하므로, 파일 디스크립터(nofile) 한도를 넉넉하게(65536) 늘려주어야 네트워크 병목이나 오류를 방지할 수 있습니다.
- `healthcheck` : 주기적으로 `tctl status` 명령을 실행해 Teleport가 정상적으로 응답하는지 컨테이너 상태를 점검합니다.

4. Teleport 서버 기동

```
1 cd /opt/teleport    # docker-compose.yml 파일이 존재하는 곳
2 docker compose up -d
3 docker compose logs -f teleport
```

정상 기동 시 보통 아래와 비슷한 로그가 보입니다.

INFO [PROXY] Starting proxy service...

INFO [AUTH] Auth service is starting...

INFO [ACME] Successfully obtained TLS certificate for your-domain.com

INFO [PROC] The Teleport proxy process is ready

5. 첫 관리자 계정 생성

```
1 # teleport 서버에서 실행해야 합니다. (docker-compose.yml 작업한 곳)
2 docker compose exec teleport tctl users add admin \
3   --roles=editor,access \
4   --logins=root,ubuntu,ec2-user
```

이 명령어의 주요 옵션은 다음과 같습니다.

- `--roles=editor,access` : 생성할 사용자에게 부여할 Teleport 내 역할(Role)입니다. `editor` 와 `access` 는 Teleport 기본 내장 역할로, 클러스터 접근 및 기본 관리 권한을 제공합니다.
- `--logins=root,ubuntu,ec2-user` : 이 사용자가 대상 서버(노드)에 SSH로 접속할 때 허용할 실제 OS 로그인 계정 이름입니다. 접속할 서버들에 존재하는 계정을 쉼표로 구분하여 적어줍니다.

출력 예시는 아래와 같습니다.

User "admin" has been created but requires a password.

Share this URL with the user to complete user setup, this link is valid for 1h:

<https://your-domain.com/web/invite/xxxxxxxxxxxxxxxx>

초대 링크를 열고 비밀번호와 OTP를 등록하면 웹 UI와 CLI 로그인에 모두 가능해집니다.

6. 로컬 PC에 `tsh` 설치 후 로그인

```
1 # macOS
2 brew install teleport
3
4 # Linux
5 curl https://goteleport.com/static/install.sh | bash -s 16.x oss
6
7 # Windows
8 # https://goteleport.com/download 에서 다운로드
```

설치가 끝났다면 로그인합니다.

```
tsh login --proxy=your-domain.com:443 --user=admin
```

실전 우회 템플릿이라면 여기서도 `tsh login --proxy=teleport.example.net:18443 --user=admin` 처럼 모의 외부 포트를 그대로 써야 합니다.

예시 출력:

Profile URL: <https://your-domain.com:443>

Logged in as: admin

Cluster: your-domain.com

Valid until: 2026-03-08 10:00:00 +0000 UTC

7. 대상 서버용 노드 토큰 발급

Teleport 서버에서 1시간짜리 조인 토큰을 생성합니다.

```
docker compose exec teleport tctl tokens add --type=node --ttl=1h
```

출력에는 보통 아래 정보가 포함됩니다.

The invite token: ab12cd34ef56gh78ij90

This token will expire in 60 minutes.

Run this on the new node to join the cluster:

```
> teleport start \  
  --roles=node \  
  --token=ab12cd34ef56gh78ij90 \  
  --ca-pin=sha256:xxxx \  
  --auth-server=your-domain.com:3025
```

실제로는 `token` 과 `ca-pin` 만 챙기면 됩니다.

8. 대상 서버에 Teleport Node Agent 설치

대상 서버에서 아래 명령을 실행합니다.

```
curl https://goteleport.com/static/install.sh | bash -s 16.x oss
```

이후 설정 파일을 생성합니다.

```
1 sudo teleport configure \  
2   --roles=node \  
3   --proxy=your-domain.com:443 \  
4   --token=ab12cd34ef56gh78ij90 \  
5   --ca-pin=sha256:xxxx \  
6   --nodename=my-server-01 \  
7   -o /etc/teleport.yaml
```

실전 우회 템플릿에서는 `--proxy=teleport.example.net:18443` 로 맞추고, 수동 템플릿을 쓸 경우 아래 `proxy_server` 값도 동일하게 바꿔야 합니다.

이 방법은 보통 운영에서 가장 다루기 편합니다. 대상 서버가 프록시를 통해 클러스터에 붙기 때문에, 개별 서버마다 별도 인바운드 SSH 포트를 열 필요가 없습니다.

라벨을 직접 주고 싶다면 설정 파일을 수동으로 써도 됩니다.

```
1 version: v3
2 teleport:
3   nodename: my-server-01
4   data_dir: /var/lib/teleport
5   auth_token: "ab12cd34ef56gh78ij90"
6   ca_pin: "sha256:xxxx"
7   proxy_server: "your-domain.com:443"
8
9   auth_service:
10     enabled: false
11
12   proxy_service:
13     enabled: false
14
15   ssh_service:
16     enabled: true
17   labels:
18     env: production
19     team: backend
20     os: ubuntu
```

라벨은 나중에 `tsh ssh ubuntu@env=production` 같은 방식으로 매우 유용하게 쓰입니다.

9. 대상 서버 서비스 활성화

```
1 sudo systemctl enable teleport
2 sudo systemctl start teleport
3 sudo systemctl status teleport
```

등록 여부는 두 방법으로 확인할 수 있습니다.

```
1 # Teleport 서버에서
2 docker compose exec teleport tctl nodes ls
3
```

```
4 # 로컬 PC에서
5 tsh ls
```

예시:

Node Name	Address	Labels

my-server-01	<tunnel>	env=production,team=backend

10. 실제 SSH 접속

기본 접속은 서버 이름으로 하면 됩니다.

```
tsh ssh ubuntu@my-server-01
```

라벨 기반 접속도 가능합니다.

```
tsh ssh ubuntu@env=production
```

여러 서버에 동시에 명령을 보낼 수도 있습니다.

```
tsh ssh --cluster=your-domain.com 'env=production' -- uptime
```

11. RBAC 역할 만들기

운영에서 중요한 건 "누가 어디까지 들어갈 수 있느냐" 입니다. 예를 들어 개발자에게는 dev, staging만 열고 production은 막고 싶다면 아래처럼 Role을 정의하면 됩니다.

```
1 kind: role
2 version: v7
3 metadata:
```

```
4   name: developer
5 spec:
6   allow:
7     logins: [ubuntu, ec2-user]
8     node_labels:
9       env: [development, staging]
10    ssh_file_copy: true
11    port_forwarding: true
12  deny:
13    node_labels:
14      env: [production]
15  options:
16    max_session_ttl: 8h
17    disconnect_expired_cert: true
```

적용과 사용자 할당:

```
1 docker compose exec teleport tctl create -f developer-role.yaml
2
3 docker compose exec teleport tctl users update developer-user \
4   --set-roles=developer
5
6 docker compose exec teleport tctl get roles
```

추가 사용자를 초대할 때도 같은 원리입니다.

```
1 docker compose exec teleport tctl users add newuser \
2   --roles=developer \
3   --logins=ubuntu,ec2-user
```

12. 세션 녹화와 감사 로그 확인

Teleport의 장점 중 하나는 SSH 접속이 끝나고 나서도 누가 무엇을 했는지 되짚어 볼 수 있다는 점입니다.

```
1 tsh recordings ls
2 tsh play <session-id>
```

웹 UI에서는 Activity → Session Recordings 메뉴에서 동일한 내용을 확인할 수 있습니다.

포트 정리

포트	기본 템플릿	DNS-01 우회 템플릿
443	Teleport Web UI, tsh login ,HTTPS, ACME	Nginx 컨테이너 내부 listen 443 , 외부로는 직접 노출하지 않음
3025	Auth 서버 조인	동일
3024	Reverse tunnel	동일
18443	보통 사용하지 않음	모의 외부 Teleport 접속 포트
18080	보통 사용하지 않음	모의 HTTP 리다이렉트 전용 외부 포트

운영에서는 Reverse Tunnel 구성을 선호하는 경우가 많습니다. 외부에서 각 서버로 직접 SSH를 열지 않아도 되기 때문입니다.

자주 쓰는 점검 명령어

문제가 생겼을 때 가장 먼저 보는 명령은 아래 정도면 충분합니다.

```
1 docker compose exec teleport tctl status
2 docker compose exec teleport tctl get cert_authorities
3 docker compose exec teleport tctl tokens ls
4 docker compose exec teleport tctl nodes ls
5 docker compose restart teleport
6 docker compose logs --tail=100 teleport
```

로컬에서는 아래 두 개도 자주 씁니다.

```
1 tsh status
2 tsh ls
```

전체 흐름 요약

1. docker compose up -d
2. tctl users add admin
3. 초대 링크로 비밀번호 + OTP 등록
4. tsh login
5. tctl tokens add --type=node
6. 대상 서버에서 teleport configure
7. systemctl start teleport
8. tsh ls
9. tsh ssh ubuntu@my-server-01

정리하면 Teleport 운영의 핵심은 세 가지입니다.

1. 사용자 로그인은 `tsh login` 으로 통일한다.
2. 서버 등록은 Node Agent와 라벨 중심으로 관리한다.
3. 접근 권한은 SSH 키가 아니라 Role과 세션 기록으로 통제한다.

여기까지 구성하면 SSH 키 파일을 서버마다 복사하고 `known_hosts` 를 손으로 맞추던 운영 방식에서 꽤 깔끔하게 벗어날 수 있습니다. 이후에는 Kubernetes, Database, GitHub SSO, Google SSO 같은 리소스를 같은 방식으로 확장해 나가면 됩니다.